Appendix 1

ENACT REFERENCE MANUAL

A1.1 GETTING STARTED

The implementation of Enact is invoked as a traditional read–eval–print loop. The user may define functions and evaluate expressions. The following shows the screen during a sample interaction with Enact.

```
C:\>enact
Enact 1.00, Copyright Peter Henderson, 1993
f(x,y):=x+y.
  aFunction
f(1,2).
  3
f(f(1,2),3).
  6
1+2*3+4*5+6.
  33
a:=new Object.
  a
a.x:=99.
  99
a.x+1.
  100
```

You will see that functions can be defined and expressions containing them can be evaluated. Note how each expression to be evaluated is terminated by a *hard full stop*, a period followed immediately by white space, usually an end-of-line. The expression will not be evaluated until the hard full stop has been read. The value of each expression

above is indented slightly from the left margin. Enact is based on expression evaluation. Expressions are built from operators and operands. Operators have a precedence which dictates the order of evaluation. For example,

```
a+b*c
```

is an expression with three operands `a`, `b` and `c`, and two binary operators plus (+) with precedence 5 and multiply (*) with precedence 4. The precedence dictates that multiplication with the higher precedence binds more tightly than addition. Thus the above expression is equivalent to

```
a+(b*c)
```

There are thirteen levels of precedence, with 1 being the highest, most binding, level. The Enact operators, each with their precedence, are listed later in the appendix. In Enact, identifiers are made from letters, digits and the underscore character. The first character of an identifier cannot be a digit. Numbers are made from digits. Negative numbers are prefixed with tilde (~*99*). Comments can be included in files following a % symbol; everything between this symbol and the next newline is ignored.

A1.2 FUNCTIONAL PROGRAMMING

As the previous section has shown, Enact is based on the idea of defining functions and calling them to effect computation. By defining functions which call others, which in turn call yet others, substantial programs can be written which use only this method of computation. This style, which avoids assignment and side-effect, is called functional programming. Enact supports a full range of functional programming concepts (with the exception of lazy evaluation) which we describe in this section. Enact also supports object-oriented programming and assignment as an extension to functional programming. These additional techniques are introduced in the following sections.

The usual definition of `factorial`, a well known example from functional programming, is written in Enact as follows:

```
factorial(n) := n=0 then 1 else n*factorial(n-1).
```

The most unusual thing is that the conditional is written using binary operators. In fact, `then` and `else` are binary operators with the property that

```
e1 then e2 else e3
```

has the meaning we usually associate with *if e1 then e2 else e3*. Because `then` and `else` are both operators we see that

```
e1 then e2 else e3 then e4 else e5
```

must have a meaning. Because of the relative priority of then and else it has the meaning *if e1 then e2 else if e3 then e4 else e5*. The operator then used alone as in `e1 then e2` means *if e1 then e2 else undefined*. Omitting the `if` may take some getting used to.

Another familiar definition is that of *nfib*, which is written in Enact as follows:

```
nfib(n) := n<2 then 1 else nfib(n-1)+nfib(n-2)+1.
```

List processing is accomplished using `hd`, `tl` and `:` as **cons**, as the following familiar definition shows. However, it should be said that list processing is below the level at which Enact is intended to be used for modelling. The object-oriented extensions provide more powerful collection types which should be used in preference.

```
reverse(x) := x=nil then nil else
                  append(reverse(tl x),(hd x):()).
```

The following interaction shows these functions being loaded, from the file **"bench.act"** and then evaluated for various arguments. This file, which is on the distribution disk, contains the three functions defined in this section.

```
load "bench.act".
  "bench.act"
  "bench.act  3:25pm Aug 28,1992"
  aFunction
  aFunction
  aFunction
  nil
x:=(1,2,3,4,5,6,7,8).
  ( 1 2 3 4 5 6 7 8 )
append(x,x).
  ( 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 )
reverse x.
  ( 8 7 6 5 4 3 2 1 )
append(x,reverse x).
  ( 1 2 3 4 5 6 7 8 8 7 6 5 4 3 2 1 )
```

These functions are used for benchmarking the implementation. For example, we evaluate

```
tm:=time(). res:=nfib 20. tm:=time()-tm. res/tm.
```

to get a raw speed figure. On the above input the system responds with the answers

```
708154325
21891
183
119
```

The four results shown are respectively the (ANSI C) time, which is seconds since the start of 1970, the value of *nfib 20*, the execution time in seconds and finally the number of calls of *nfib* per sec. In this version of the implementation this figure is rather lower than I would wish it to be. Later versions will be faster. For those familiar with functional programming, functions are first-class citizens and hence currying and other higher order methods work properly, as the following definitions show:

```
inc x := x+1; twice f x := f(f x); twice twice twice inc 0.
  16
```

Various predefined functions are available. They are dealt with in a later section.

A1.3 OBJECT-ORIENTED PROGRAMMING

There is a simple object-oriented component which integrates quite well with the functional programming. There is a predefined object `Object`, which will be the root of the class hierarchy. We can define objects which will have a class and we can define new classes in terms of existing ones. Every class is represented in the system by an object. To define a new class we link it into the object hierarchy explicitly. For example, the following assignments create three new classes, and link them into the class hierarchy.

```
class Node < Object.
class Leaf < Node.
class Tree < Node.
```

`Tree` and `Leaf` are subclasses of the class `Node`, which in turn is a subclass of the class `Object`. We use `new` to generate instances of these classes. Thus `new Tree` is an expression which denotes a new object of class `Tree`. Objects have attributes. The expression `new Leaf with value=99` denotes an object of class `Leaf` with an attribute `value` set to `99`. So far this is all entirely functional: there are no side-effects. If `x` is an object, then `x.value` denotes the value of its `value` attribute. With this preparation, look at the following definitions:

```
leaf(aNumber):=new Leaf with value=aNumber.

Leaf.sum():=self.value.

tree(aNode1,aNode2):=new Tree with left=aNode1
                          with right=aNode2.

Tree.sum():=self.left.sum()+self.right.sum().

t:=tree(tree(leaf(99),leaf(100)),leaf(101)).
```

Here we have defined methods for each of our classes and also constructor functions. The expression `tree(tree(leaf(99),leaf(100)),leaf(101))` constructs a binary tree with three leaves. The call `t.sum()` computes the value `300` by summing over these nodes. Because the function sum has been defined as `Leaf.sum` and `Tree.sum`, we refer to it as a *method*. It must be invoked by calling it as `n.sum`, where `n` is a `Node` (i.e. either a `Leaf` or a `Tree`). In the body of a method, the variable `self` refers to the object upon which the method was originally invoked. Thus, if `t` is a `Tree` then, when `t.sum()` is invoked, the occurrences of `self` in the body of the corresponding method refer to `t`. The use of variable names of the sort `aNode`, `aTree` etc. is an idiom borrowed from Smalltalk which is very readable.

A1.4 ASSIGNMENT

You will have seen the use of assignment at the topmost level. We have also seen that it is possible to nest assignments, assigning to local or global variables or to the attributes of objects. These two sorts of assignment have subtly different meanings. The following interactions show what is possible.

```
x:=1; y:=2; (x,y).
  ( 1 2 )
program(x):=(x:=x+10; y:=y+10; (x,y)).
  aFunction
x:=3; y:=4; (x,y).
  ( 3 4 )
program(5).
  ( 15 12 )
(x,y).
  ( 3 4 )
```

The final value of `y` illustrates that the variables in `program` are local. Both `x` and `y` are local variables which are initialized, respectively, to the value of the parameter of `program` and to the value of the global `y` at the time the function is defined. This initialization is repeated each time `program` is called. Although the `y` inside `program` has the value `12` at the end, that is not the value of the global `y` after the call. Suppose, however that `y` is an object:

```
y:=new Object.
  y
x:=1; y.value:=2; (x,y.value).
  ( 1 2 )
program(x):=(x:=x+10; y.value:=y.value+10; (x,y.value)).
  aFunction
x:=3; y.value:=4; (x,y.value).
  ( 3 4 )
program(5).
```

```
  ( 15 14 )
(x,y.value).
  ( 3 14 )
```

The explanation is that variables inside functions behave as initialized local variables, parameters taking their initialization from actual arguments and free variables their initialization from the value they had at function definition time. This is true even of object-valued variables. But objects themselves are global. Assignment to their attributes is therefore globally felt.

Let us repeat the *Tree* example, this time using assignment, with the effect that when we assign to the value of a leaf all of the values of the nodes in the tree are recomputed. First we change the constructors so that each *Node* has a *value* attribute and a *parent* attribute. Then we ensure that when a tree with children is constructed, the *parent* attribute is correctly set.

```
leaf(aNumber):=new Leaf with value=aNumber
                        with parent=nil.

tree(aNode1,aNode2):=
  (t:=new Tree with left=aNode1
                with right=aNode2
                with value=aNode1.value+aNode2.value
                with parent=nil;
   aNode1.setParent(t); aNode2.setParent(t);
   t).

Node.setParent(aNode):=(self.parent:=aNode).
```

We can still construct a tree as before, but now the value of each node will be computed as the tree is constructed. For example,

```
leaf1:=leaf(99); leaf2:=leaf(100); leaf3:=leaf(101);
t:=tree(tree(leaf1,leaf2),leaf3).
```

constructs the same tree as before, but with the added convenience of introducing some variables which we can use to refer to each node. It is interesting to use Enact just to navigate this structure, as follows:

```
leaf1.value.
  99
t.value.
  300
t.right.
  leaf3
t.left.
  aTree
```

```
leaf3.parent.
  t
```

Note that when Enact is required to print a structure (an object) it uses the name of a global variable, if such exists (e.g. `leaf3` above), otherwise it makes up a name, such as `aTree`, by concatenating `a` or `an` to the class name.

Now we define two methods which allow us to set the value of a leaf and recompute the value of all its parent and grandparent (etc.) nodes.

```
Leaf.setValue(aNumber):=
  (self.value:=aNumber;
   self.parent<>nil then self.parent.compute()).

Tree.compute():=
  (self.value:=self.left.value+self.right.value;
   self.parent<>nil then self.parent.compute()).
```

Note that both methods inspect the parent link, and if it is set, call `compute` to recompute the parent value. The following interaction shows these methods in use.

```
leaf1.value.
  99
t.value.
  300
leaf1.setValue(102); t.value.
  303
```

A1.5 INHERITANCE

In the previous sections we have made some elementary use of inheritance. Since `Leaf` and `Tree` are subclasses of `Node`, they inherit the attributes of `Node`. For example, the method `Node.setParent(aNode)` is inherited.

Enact supports multiple inheritance. When introducing a new class one can specify a number of superclasses, as the following example shows. First we define two classes with no superclasses:

```
class A < ().
  A
A.f():=99.
  aFunction
class B < ().
  B
B.g():=100.
  aFunction
```

Then we define a class which has two superclasses.

```
class C < (A,B).
  C
```

Now we can define an object of class `C` and expect it to inherit from both `A` and `B`:

```
c:=new C.
  c
c.f().
  99
c.g().
  100
```

If we revise the definition of `A.f` as follows

```
A.f():=self.g()+99.
  aFunction
c.f().
  199
```

we see that it correctly calls `B.g` when invoked as `c.f()`.

A1.6 OPERATOR PRECEDENCE

All operators in Enact have a numerical precedence. There are thirteen levels of precedence, with 1 being the highest, most binding, level. The Enact operators with their precedence are as follows:

```
1  ' new
2
3  . application
4  * / mod
5  + - :
6  = > < >= <= <>
7  with :: where
8  and loop
9  or
10 if then
11 else
12  :=
13  ; class load fix
```

All operators are either unary or binary. Binary operators at the same level of precedence associate to the left. Thus `1-2-3-4` means `((1-2)-3)-4`. We shall list the operators here and illustrate each of them in use in the following sections.

Arithmetic operators `a+b`, `a-b`, `a*b`, `a/b`, `a mod b`, (there is no unary minus). These operators have their usual relative precedence. They compute integer values using 32 bit arithmetic.

Relational operators `a=b`, `a<>b`, `a<b`, `a>b`, `a<=b`, `a>=b`. These operators have their usual relative precedence and compute logical values `true` and `false`. Lower precedence than arithmetic ensures that expressions like `i+1<2*j` parse correctly.

Logical operators `a and b` and `a or b` have their usual relative precedence and meaning. Negation is defined as a unary function `not` and so has precedence `3`. This means its argument usually requires to be parenthesized.

Function definition `f(x):=x+1` and `f x:=x+1` are identical in meaning. They both define the function `f` of one argument. Functions of more than one argument have their arguments in parentheses `g(x,y):=x+y`.

Function application `f(99)` and `f 99` are identical in meaning. Functions of more than one argument are applied as `g(1,2)`. Precedence is higher than arithmetic operators, so that `f(99)+g(1,2)` will parse correctly. Free variables are statically bound at time of definition.

Lambda expression `x::x+1`, `(x)::x+1` are lambda expressions and identical in meaning. Functions of more than one argument are defined as `(x,y)::x+y`. Precedence of `::` is lower than arithmetic and relational operators, but not logical operators, so that `(x,y)::(x<=y and y<=x+1)` will parse correctly only because of the parentheses around the body. An alternative way to define functions is `f:=x::x+1`, `f:=(x)::x+1` or `g:=(x,y)::x+y`, but this is not good style.

Currying This is allowed, as are other higher order devices. If we define `h x y:=x+y` then `h 1 2` evaluates to `3` because `h 1` evaluates to `y::1+y`.

Local definitions `x+y where x=99` and `x+y where (x,y)=(99,100)` have their normal meaning.

Conditional expressions These are constructed using the operators

```
a then b
b if a
c else d
```

Both `a then b` and `b if a` have the value `b`, assuming `a` evaluates to `true`, otherwise they have the value *undefined* (so in fact, `b if a` is just an alternative form of `a then b`). The expression `c else d` has the value `c` unless this value is *undefined*, in which case it has the value `d`. Precedence ensures that, for example, `a then b else d` has the expected meaning.

Lists These are constructed using the operator `a:b`. There are predefined functions `hd c`, `tl c`. The expression `(x,y,z,...)` evaluates its arguments and constructs a list.

The expression `nil` is the empty list and `a:b` is the list whose head is `a` and whose tail is `b`. Precedence ensures that, for example, `hd a:b` has the meaning `(hd a):b`. The expression `atom c` is false if `c` is a list, unless `c=nil`. Non-numeric atoms are constructed with a preceding single quote, e.g. `'hello` or `'"hello world"`. Double quotes are required for atoms which are not simple identifiers.

Singleton list Because Enact *always* takes a set of parentheses to be a bracketing structure, which, apart from their effect on precedence, can be ignored, it is difficult to denote a singleton list. The expression `(99)` is equivalent to `99`. To denote a singleton list we must write `99:()` or `99:nil` or `list 99`.

Assignment statements The meaning of `x:=e` is to assign the value of `e` to the local variable `x`. Inside functions, free variables and parameters are local to the body. Thus assignment to a variable is local to the body of that function. An assignment statement is an expression. Its value is the value of `e`. The precedence of `:=` is very low (`12`) so care must be taken. For example in `p then (x:=y) else (x:=0-y)` the parentheses are essential. Similarly `(x:=y) if p` requires the parentheses around the assignment statement to avoid the interpretation `x:=(y if p)` which arises because `if` is more binding than `:=`.

Other statements The meaning of `S;T` is evaluate `S` for its effect, then evaluate `T`. The iterative statement `e loop S` repeats `S` while `e` returns true. Looping in this way should be avoided, in preference for iterators over structures.

Classes `class A < B` introduces a new class `A`, as a subclass of `B`. There is a root class `Object`. If only single inheritance is being used all new classes should be directly or indirectly a subclass of `Object`. The expression `class A < (B,C,D)` introduces a new class `A`, as a subclass of `B`, `C` and `D`.

Objects The expression `new A` creates a new object of class `A`. The more elaborate form `new A with attr1=e1 with attr2=e2 ...` creates an object with some of its attributes initialized.

Assignment to attributes After `a:=new A` we can assign attributes with `a.attr1:=e1` and we can access attributes with `a.attr1`. Both classes and objects can have attributes.

Methods These are just function valued attributes of classes, for example `A.f(x,y):=x+y+self.attr1`. If this method is invoked using `a.f(1,2)` then in its body `self` has the value `a`.

Fixed point Mutually recursive definitions can be made using the *fix* operator. For example, `(P,Q)fix(P:= ...; Q:= ...)` will assign values to `P` and `Q` where each is defined in terms of the other. The actual values of these two variables are available during the evaluation of the ellipsis in a restricted way. Usually, this operator is used only to define mutually recursive functions, as in `(f,g)fix(f(x):=... g(x) ...; g(x):= ... f(x) ...)`.

Loading files the expression `load "filename"` will read a script from the file with name *filename*. This is the usual way of storing an Enact model and loading it into the interpreter for testing. A full path name can be given.

A1.7 PREDEFINED FUNCTIONS AND CLASSES

The following functions are predefined:

`not x`	Boolean negation
`hd x`	first item of list
`tl x`	all but first item of list
`atom x`	true if *x* is a number or a symbol, false if *x* is a list, if *x* is nil, if *x* is a function or if *x* is an object
`isObject x`	true if *x* is an object, otherwise false
`append(x,y)`	append lists *x* and *y*
`size x`	returns length of list *x*
`map(f,y)`	apply function *f* to each element in the list *y*
`filter(f,y)`	select those elements of list *y* for which function *f* returns true
`all(f,y)`	true if applying function *f* to each element in the list *x* yields true
`reduce(g,u,x)`	compute `g(x1,g(x2, ... g(xn,u)...))`. For example, `reduce((x,y)::x+y,0,(1,2,3,4,5,6,7,8,9))` computes `1+2+3+4+5+6+7+8+9+0`
`union(x,y)`	assuming *x* and *y* are lists without repetition (pretending to be `sets`), form the set union of lists *x* and *y*. Used to implement object-oriented set operations (defined later)
`difference(x,y)`	assuming *x* and *y* are lists without repetition (`sets`), form the set difference of lists *x* and *y*. Used to implement object-oriented set operations (defined later)
`intersection(x,y)`	assuming *x* and *y* are lists without repetition (`sets`), form the set intersection of lists *x* and *y*. Used to implement object-oriented set operations (defined later)

`member(x,y)`	assuming *y* is a list without repetition (`set`), return true only if *x* is a member of list *y*. Used to implement object-oriented set operations (defined later)
`remove(x,y)`	assuming *y* is a list without repetition (`set`), return list with same members as *y* except for one occurrence of *x* (if any). Used to implement object-oriented set operations (defined later)
`classof x`	returns class of object *x*
`attrs x`	returns immediate attributes of object *x* (not inherited attributes)
`time()`	(ANSI C) time, seconds since 00:00:00 GMT 1 Jan 1970 (approx.)
`cells()`	number of list cells currently in use
`maxcells()`	maximum number of list cells in use this session
`version()`	version of Enact in use
`bye()`	leave Enact (or use Ctrl-C)
`ask x`	input–output, prompts with message *x*, returns atom typed by user as value of function (e.g. `age:=ask '"how old are you".`) This operation should be used sparingly.

There are also predefined collection classes, specifically `Set` and `Bag`, with the following meanings. Their full definitions are included in Appendix 2. They are also to be found in the file **`enact.cfg`**.

Consider the following interaction, which illustrates the use of some of these collection classes.

```
s:=set(); s.insert(1); s.insert(2); s.insert(3).
  ( 3 2 1 )
s.members.
  ( 3 2 1 )
t:=s.collect(x::x+2).
  t
t.members.
  ( 5 4 3 )
s.union(t).members.
  ( 2 1 5 4 3 )
s.difference(t).union(t.difference(s)).members.
  ( 2 1 5 4 )
```

```
u:=unitset(s).union(unitset(t)).
  u
u.members.
  ( s t )
u.UNION().members.
  ( 2 1 5 4 3 )
```

Once the set `s` is constructed we must use `s.members` to see its members (as a list). The expression `s.collect(x::x+2)` computes a new set whose members are determined from the members of `s` using the function `x::x+2`. The `collect` and `select` methods are named after the same methods in Smalltalk. As you can see from their definitions (Appendix 2) they are the operations familiar to functional programmers as *map* and *filter*.

The expression `s.union(t)` computes the set union of `s` and `t`. The more complex expression `s.difference(t).union(t.difference(s))` computes those elements in either `s` or `t`, but not both. It parses correctly because function application and dot have the same precedence.

The remainder of the above example shows the construction of a set of sets `u`. Enact displays this as the list `( s t )`, the printing of the identifiers here standing for the actual sets which are members of `u`. The method `Set.UNION()` always applies to a set of sets, which it flattens by forming the union of all the member sets.

Both classes `Set` and `Bag` are subclasses of the class `Collection`. The following constructors and methods are defined.

`set()` constructor, returns a new empty set

`bag()` constructor, returns a new empty bag

`Collection.member(anObject)` true only if `anObject` is a member of the collection

`Collection.size()` returns the number of members in the collection

`Collection.insert(anObject)` changes the collection to include the member `anObject`. If the collection is a set then repetition is avoided.

`Collection.remove(anObject)` changes the collection to remove the member `anObject`. If the collection is a bag then all repetitions are removed.

`Collection.collect(aFunction)` constructs a new collection of the same type, whose members are obtained from the collection by applying `aFunction` to each of its members

`Collection.forEachDo(aFunction)` applies `aFunction` to each of the collection's members, purely for the side-effect that the function has.

`Collection.select(aFunction)` constructs a new collection of the same type, whose members are obtained from the collection by selecting only those for which `aFunction` returns true

`Collection.reduce(aFunction)` applies the binary function `aFunction` to the collection thus reducing it to a single item. Assumes the collection is not empty

`Collection.locate(aFunction)` returns an item from the collection by selecting one for which `aFunction` returns true. Assumes such an item exists

`Collection.all(aFunction)` applies `aFunction`, which returns true or false to every item in the collection. Returns true only if all items return true

`Collection.exists(aFunction)` applies `aFunction`, which returns true or false to every item in the collection. Returns true if any item returns true

`Set.add(anObject)` constructs a new Set, whose members are those of the given set with `anObject` added

`Set.subset(aSet)` true if set is a subset of `aSet`

`Set.equal(aSet)` true if set is equal to `aSet`

`unitset(anObject)` constructs a set with one member

`Set.union(aSet)` constructs a set which is the union of the two given sets

`Set.difference(aSet)` constructs a set which is the difference of the two given sets

`Set.intersection(aSet)` constructs a set which is the difference of the two given sets

`Set.UNION()` assumes the set is a set of sets, applies union to flatten this into a single set containing all the members of each member set

The final items in the Enact library are predefined functions for determining the inheritance relationships between classes and for determining which attributes are inherited from where.

`attributes obj` the immediate attributes of object `obj`, presented as a `Set`

`classes obj` the immediate classes of object `obj`, presented as a `Set` (usually a unit set)

`supers c` the immediate superclasses of class `c`, presented as a `Set`

`superiors c` all the superclasses of class `c`, presented as a `Set`

`suppliers(obj,attr)` all the classes which supply attribute `attr` to object `obj`, presented as a `Set`. The `attr` argument must be quoted (e.g. `suppliers(m,'do)`)

alphabet obj all the attributes, including inherited attributes, of object *obj*, presented as a Set

OK obj true if all the attributes, including inherited attributes, of object *obj*, are inherited from exactly one superclass

badAttrs obj those attributes, including inherited attributes, of object *obj* which are inherited from more than one superclass, presented as a Set

As an example of the use of these functions, consider the following interaction:

```
attributes(M).members.
  ( objects labeller )
alphabet(M).members.
  ( addDependent doAllDependents do setLabel
      pick install objects labeller )
classes(M).members.
  ( Menu )
superiors(Menu).members.
  ( Event Scroller Object )
suppliers(M,'install).members.
  ( Scroller )
OK M.
  true
Event.install():=99.      % just to illustrate problems
  aFunction
OK M.
  false
badAttrs(M).members.
  ( install )
suppliers(M,'install).members.
  ( Event Scroller )
```

In fact the hierarchy here is that *Menu* is a subclass of both *Event* and *Scroller*. Initially each attribute is inherited uniquely. But by adding an *install* attribute to *Event* we can see the use of *OK*, *badAttrs* and *suppliers* to isolate the problem.

An attribute which is inherited from two different superclasses (either directly or indirectly), if it is not an error, needs disambiguation. This is achieved by redefining it in the common child so that the two inherited occurrences are masked.

The definitions of these predefined functions show something of the power of Enact when used to model complex structures. They also illustrate an unusual implementation feature of Enact, that in fact there is no distinction (in the implementation) between 'is an instance of' and 'is a subclass of'. However, the distinction can be maintained by the user of Enact, by following the style of use illustrated earlier in this manual.

These definitions will be found in **enact.cfg**.

```
attributes x := new Set with members = attrs x.

supers x := new Set with members =
 (isObject(classof x) then list(classof x) else classof x).

classes := supers.
superiors x :=
 supers(x).union(supers(x).collect(superiors).UNION()).

suppliers(obj,attr):=
 attributes(obj).member(attr) then unitset(obj) else
   supers(obj).collect(aClass::suppliers(aClass,attr)).UNION().

alphabet obj :=
 superiors(obj).collect(attributes).UNION().union(attributes obj).

OK obj :=
 alphabet(obj).all(anAttr::suppliers(obj,anAttr).size()=1).

badAttrs obj :=
 alphabet(obj).select(anAttr::suppliers(obj,anAttr).size()<>1).
```

A1.8 SOME ISSUES

One of the main concepts of object-oriented design, and indeed of good design in general, has been omitted. Encapsulation of abstractions, specifically abstract data types, has not been addressed. Far from it: all classes expose all their attributes to everyone.

This was not an oversight. I am trying not to be too prescriptive in Enact about how one should organize and present abstractions. Enact is a toolkit which can be used in various ways, which I hope to show in additional papers. For example it can be used to exercise state-based specifications or to present algebraic ones. I have used it to define a CSP interpreter and then used that interpreter to test a specification of a protocol.

I felt that the organization of all designs into simple abstract data types was too constraining, even though most of all the designs I have done do indeed fit this model. Enact users are expected to adopt design constraints suitable to their application areas and then apply a judicious amount of self-discipline when representing their designs in Enact. Some of the examples I have done illustrate the variety of such constraints and will be published in the fullness of time. In particular, a method based on developing collections of related abstract data types, which I call abstract systems, is the subject of this book. I show how such models can be implemented quite effectively in C++.

Another issue which Enact is quite liberal about is binding. Ordinary variables are bound statically. Attributes are bound dynamically. The static binding of variables extends to all free variables in the body of a function. When a function is defined its free variables are bound to the value they have at definition time. Assignment to such

variables is not an assignment to a global variable with the same name. It is an assignment to the (automatically declared) local variable which has been initialized to the value of the corresponding global. Consequently, functions which assign only to statically bound variables (i.e. not attributes) are purely functional, they can have no side-effect.

The assignment within a function to the attribute of a global object is, however, felt as a side-effect. The global variable remains bound to the same object, but the value of that object's attribute changes. This liberal attitude to side-effects seems necessary for some kinds of modelling.

Finally, a word about sets. Of course these are not (exactly) mathematical sets. For a start, they must be finite (in fact they had better be quite small). But there are other ways in which they do not behave exactly as they should if they were mathematical sets. The most important is that when comparing members (for example when computing union) the equality operation used is what is usually referred to as pointer-equality. Simple values (numbers and symbols) are of course recognized as equal. But objects which may be equal, in the sense that any attempt to interrogate them through their methods would yield identical results, can nevertheless be unequal if they have been constructed separately and are thus represented by different pointers.

A1.9 AVAILABILITY

In addition to the implementation provided in the book, Enact implementations are available free of charge (or for the cost of distribution, if any) from Peter Henderson. There is a PC version and versions for UNIX™ on Sun3 and SparcStations. You may obtain these versions electronically as described below, or copy them from someone else without further permission. If you wish to be kept informed about Enact developments then please let me know, preferably by email. I am **peter@ecs.soton.ac.uk** . Please report any bugs you find.

To obtain copies of the implementation electronically you can make use of anonymous FTP. To do this you must be able to access internet with a suitable FTP program. Most universities worldwide will be able to provide such access. Use FTP to gain access to my site **ecs.soton.ac.uk** and then log on with the username **anonymous**. When asked for a password, enter your full email address (yes, give your address as the password). You should now be able to locate the Enact implementation. It is currently located in the directory **pub/peter** where a file called **README** explains the status of the other files in that directory. If this does not work, email me for advice.